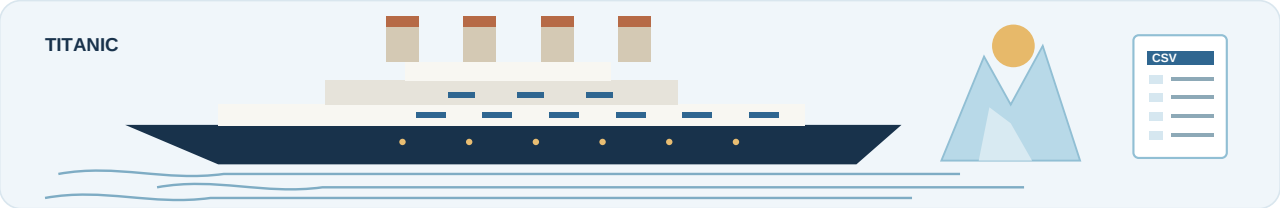
Titanic Survivor Data Preparation Pipeline

Minor Project | Machine Learning Data Preparation

Name: Sri Santhosh R

PROJECT AREA	DELIVERABLE	DATASET
Data cleaning and feature preparation	Runnable Python script and prepared CSV outputs	Seaborn Titanic example dataset



Project overview

This project prepares the Titanic passenger dataset for downstream machine learning. A maritime historian needs clean, encoded, scaled, and split data for later survival analysis. The pipeline validates and cleans inputs, removes leakage and redundant fields, handles missing values, creates a stratified 80/20 split, encodes categories, and scales features. It stops before model training and saves reusable feature/label CSV files.

Objectives

Preserve the survival labels, avoid target leakage, and make preprocessing reproducible for unseen data. The complete implementation appears in the appendix; no prediction model is trained or evaluated.

1. Dataset and preparation design

The source is Seaborn's example dataset named **titanic**. Its target is **survived** (0 = did not survive, 1 = survived). The preparation uses passenger and voyage attributes such as class, sex, age, family counts, fare, and embarkation. Seaborn downloads the data from its example-data repository when it is not cached, so first-time use requires an internet connection.

Field	Treatment	Reason
survived	Kept separately as y	Target label for later supervised learning
alive	Removed	Text duplicate of the target; would leak the answer
deck	Removed	Mostly missing and not needed for this exercise
class / embark_town	Removed	Duplicate representations of pclass / embarked
sex / alone / adult_male	Binary numeric	Two-level fields become numeric indicators
pclass	Retained as ordinal numeric	Passenger classes have a natural order
embarked / who	One-hot encoded	Nominal categories have no numeric order

Data handling note

The train/test split occurs before imputation, category encoding, and scaling. The test set is transformed using rules learned from the training set, so test observations do not influence medians, modes, one-hot columns, means, or standard deviations. This ordering preserves a fair downstream evaluation.

2. Pipeline steps

Step	Function	Action
1	load_data()	Load the Seaborn Titanic table. A clear error explains that dataset access may need internet.
2	validate_data()	Require the target and key input fields, reject empty data, null targets, and unexpected target values.
3	clean_text()	Trim whitespace and lowercase string categories without changing numeric or boolean columns.
4	drop_useless()	Remove alive (target leakage), deck (mostly missing), and duplicated class/embark_town fields.
5	split_data()	Create an 80/20 split with random_state=42 and stratify=survived, preserving class proportions.
6	handle_missing()	Use training medians for numeric columns and training modes for categorical columns; reuse them on test.
7	encode_categoricals()	Map binary fields, retain ordinal pclass, and one-hot encode embarked and who. Align test columns to training.
8	scale_features()	Fit StandardScaler on training inputs, then transform train and test using the same parameters.
-	main()	Call functions in order, save the two CSVs, and report validation/runtime failures in a readable way.

Why scaling is included

Standardization converts each numeric feature using z = (x - mean) / standard deviation. It helps many downstream algorithms that are sensitive to feature scale. Some tree-based models do not require it, but the assignment asks for a scaled dataset, so the same prepared output can support scale-sensitive methods.

3. Uses and limitations

Uses

- Provides consistent input for later classification experiments such as logistic regression, nearest neighbors, or support vector machines.
- Demonstrates a reusable preprocessing workflow with a target-preserving split.
- Produces tabular CSVs that can be inspected, shared, and passed to downstream training code.
- Shows how to guard against leakage from both direct target copies and fitted preprocessing statistics.

Limitations

This is an educational preparation pipeline, not a validated safety-critical or historical inference system. The example dataset is small and has known collection and representativeness limits. Removing deck discards potentially useful information because it is heavily incomplete. Mode imputation and one-hot encoding are straightforward baseline choices, not universally optimal. The script does not persist preprocessing objects; for production inference, save a fitted pipeline so new records receive exactly the same transformations.

4. Setup and execution

Requirements

Python 3.9 or newer is recommended. Install pandas, seaborn, and scikit-learn in the active Python environment:

```
python -m pip install pandas seaborn scikit-learn
```

Run

```
python titanic_preparation_pipeline.py
```

The source script and current directory must be writable so the CSV files can be saved. If dataset download fails, check the connection or adapt load_data() to read a local Titanic CSV with pandas.read_csv(). The local file must contain the required columns listed in validate_data().

Expected result

The console prints the row counts and prepared feature count, and creates titanic_train_prepared.csv and titanic_test_prepared.csv. The precise feature count depends on categories present in the downloaded dataset. The output target column remains integer-valued; transformed features are standardized numeric values.

5. Resources used

Purpose	Resource	Reference
Dataset loader	Seaborn load_dataset API	https://seaborn.pydata.org/generated/seaborn.load_dataset.html
Dataset repository	Seaborn example data (Titanic CSV)	https://github.com/mwaskom/seaborn-data/blob/master/titanic.csv
Train/test split	scikit-learn train_test_split	https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
Feature scaling	scikit-learn StandardScaler	https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
Categorical encoding	scikit-learn preprocessing guide	https://scikit-learn.org/stable/modules/preprocessing.html

Libraries

pandas is used for tabular data and CSV export; seaborn loads the Titanic sample; scikit-learn supplies the stratified split and StandardScaler. Python standard-library modules provide exception handling and type annotations.

6. Conclusion

The project turns a raw Titanic sample into clean, encoded, scaled training and test datasets while keeping the survival label available for downstream learning. Its central design choice is to split first and fit preprocessing steps on training data only. The result is a compact, modular example of responsible data preparation; no prediction model or performance claim is included.

Appendix A Complete Python Code

This is the complete runnable script delivered with the report. Save the appendix code as titanic_preparation_pipeline.py, or use the separate Python file included with this project.

```
"""Prepare the Seaborn Titanic dataset for downstream machine learning.
This project prepares features and labels; it does not train a classifier.
Run with: python titanic_preparation_pipeline.py
"""
from __future__ import annotations
from typing import Tuple
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
TARGET = "survived"
RANDOM_STATE = 42
class DataValidationError(Exception):
    """Raised when the source data does not have the expected structure."""
def load_data() -> pd.DataFrame:
    """Load Seaborn's Titanic example dataset (download may require internet)."""
    try:
        return sns.load_dataset("titanic")
    except Exception as exc:
        raise RuntimeError(
            "Could not load the Titanic dataset. Check internet access or place "
            "a compatible titanic.csv in the working directory and load it with "
            "pandas.read_csv()."
        ) from exc
def validate_data(data: pd.DataFrame) -> None:
    """Check required fields and basic target integrity."""
    required = {TARGET, "sex", "pclass", "age", "fare", "embarked"}
    missing = sorted(required.difference(data.columns))
    if missing:
        raise DataValidationError(f"Required columns are missing: {missing}")
    if data.empty:
        raise DataValidationError("The dataset contains no rows.")
    if data[TARGET].isna().any():
        raise DataValidationError("The target column 'survived' contains nulls.")
    if not set(data[TARGET].unique()).issubset({0, 1}):
        raise DataValidationError("The target must contain only 0 and 1.")
def clean_text(data: pd.DataFrame) -> pd.DataFrame:
    """Trim whitespace and normalize text categories without changing numbers."""
    cleaned = data.copy()
    for column in cleaned.select_dtypes(include=["object", "string"]).columns:
        cleaned[column] = cleaned[column].map(
            lambda value: value.strip().lower() if isinstance(value, str) else value
        )
    return cleaned
def drop_useless(data: pd.DataFrame) -> pd.DataFrame:
    """Remove target leakage, mostly-empty, and redundant columns."""
    # 'alive' restates survived. 'deck' is mostly missing in this dataset.
    # class duplicates pclass; embark_town duplicates embarked.
    return data.drop(
        columns=["alive", "deck", "class", "embark_town"], errors="ignore"
    ).copy()
def split_data(
    data: pd.DataFrame,
) -> Tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]:
    """Create a reproducible, stratified 80/20 split before fitting transforms."""
    features = data.drop(columns=[TARGET])
    labels = data[TARGET].astype("int64")
    return train_test_split(
        features,
        labels,
        test_size=0.20,
        random_state=RANDOM_STATE,
        stratify=labels,
    )
def handle_missing(
    train: pd.DataFrame, test: pd.DataFrame
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    """Fill numeric nulls with training medians and categories with training modes."""
    train_filled, test_filled = train.copy(), test.copy()
    for column in train.columns:
        if pd.api.types.is_numeric_dtype(train[column]):
            fill_value = train[column].median()
        else:
            modes = train[column].mode(dropna=True)
            fill_value = modes.iloc[0] if not modes.empty else "unknown"
        train_filled[column] = train[column].fillna(fill_value)
        test_filled[column] = test[column].fillna(fill_value)
    return train_filled, test_filled
def encode_categoricals(
    train: pd.DataFrame, test: pd.DataFrame
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    """Binary-map two-level fields and one-hot encode nominal categories."""
```

```
    train_encoded, test_encoded = train.copy(), test.copy()
    for column in ("sex", "alone", "adult_male"):
        if column in train_encoded:
            # Values are normalized by clean_text; map booleans explicitly too.
            values = train_encoded[column]
            if values.dtype == bool:
                train_encoded[column] = values.astype("int8")
                test_encoded[column] = test_encoded[column].astype("int8")
            elif column == "sex":
                mapping = {"female": 0, "male": 1}
                train_encoded[column] = values.map(mapping).astype("int8")
                test_encoded[column] = test_encoded[column].map(mapping).astype("int8")
            else:
                train_encoded[column] = values.astype("int8")
                test_encoded[column] = test_encoded[column].astype("int8")
    nominal = [c for c in ("embarked", "who") if c in train_encoded.columns]
    if nominal:
        train_encoded = pd.get_dummies(train_encoded, columns=nominal, dtype="int8")
        test_encoded = pd.get_dummies(test_encoded, columns=nominal, dtype="int8")
    # Test categories unseen in training are ignored; missing train columns are zero.
    test_encoded = test_encoded.reindex(columns=train_encoded.columns, fill_value=0)
    return train_encoded, test_encoded
def scale_features(
    train: pd.DataFrame, test: pd.DataFrame
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    """Standardize numeric features, fitting means and scales on training data only."""
    scaler = StandardScaler()
    train_scaled = pd.DataFrame(
        scaler.fit_transform(train), columns=train.columns, index=train.index
    )
    test_scaled = pd.DataFrame(
        scaler.transform(test), columns=test.columns, index=test.index
    )
    return train_scaled, test_scaled
def main() -> None:
    """Run the complete preparation pipeline and save model-ready CSV files."""
    try:
        data = load_data()
        validate_data(data)
        data = clean_text(data)
        data = drop_useless(data)
        X_train, X_test, y_train, y_test = split_data(data)
        X_train, X_test = handle_missing(X_train, X_test)
        X_train, X_test = encode_categoricals(X_train, X_test)
        X_train, X_test = scale_features(X_train, X_test)
        train_output = X_train.copy()
        train_output[TARGET] = y_train
        test_output = X_test.copy()
        test_output[TARGET] = y_test
        train_output.to_csv("titanic_train_prepared.csv", index=False)
        test_output.to_csv("titanic_test_prepared.csv", index=False)
        print("Pipeline completed successfully.")
        print(f"Training rows: {len(train_output)} | Test rows: {len(test_output)}")
        print(f"Prepared feature count: {X_train.shape[1]}")
        print("Saved titanic_train_prepared.csv and titanic_test_prepared.csv")
    except (DataValidationError, RuntimeError) as exc:
        print(f"Pipeline could not complete: {exc}")
    except Exception as exc:
        print(f"Unexpected pipeline error: {type(exc).__name__}: {exc}")
if __name__ == "__main__":
    main()
```